


# Rapport de sécurité

La traçabilité agricole, de la parcelle au marché.

Version du 2026-10-21

# Sécurité — rapport d'analyse, correctifs et risques restants

Analyse réalisée sur l'ensemble de la pile : API (NestJS/PostgreSQL), portail web, application mobile Android, application de bureau Windows, mises à jour et chaîne de livraison. Chaque correctif est couvert par un test automatique (`npm run test:e2e:securite`, 43 contrôles, exécuté en mode production en intégration continue ; contrôles de signature dans `e2e-app-update.js` et `desktop/test/updater.js`).

**Ce rapport n'est pas un audit indépendant.** Il décrit ce que nous avons trouvé, corrigé et testé nous-mêmes. Un test d'intrusion par un tiers reste indispensable avant l'exploitation réelle.

## 1. Failles trouvées et corrigées

| # | Faille | Gravité | Correctif |
|---|---|---|---|
| 1 | Connexion : réponses et durées différentes selon que le compte existe, est suspendu ou verrouillé (énumération de comptes) | Élevée | Même message « Identifiants invalides » et même temps de calcul (empreinte factice argon2id) dans tous les cas ; tentative sur compte verrouillé journalisée |
| 2 | Aucune limite de tentatives de connexion par adresse (force brute, bourrage d'identifiants) | Élevée | 10 connexions/min/adresse (`LOGIN_THROTTLE_LIMIT`), 30 renouvellements/min ; verrouillage du compte après 5 échecs / 15 min |
| 3 | Mots de passe faibles acceptés | Élevée | Politique : ≥ 10 caractères, ni courant, ni suite évidente, ni l'e-mail, au moins 3 types de caractères (ou ≥ 16) ; appliquée à la création, au changement, à l'amorçage |
| 4 | Corps de requête non bornés sur les routes anonymes (déni de service mémoire) | Élevée | 2 Mo par défaut ; 10/40 Mo uniquement pour synchronisation, réplication, géodonnées ; 413 au-delà |
| 5 | Documentation Swagger exposée en production | Moyenne | Désactivée en production (`ENABLE_API_DOCS=true` pour la réactiver) |
| 6 | Réponses d'API susceptibles d'être mises en cache (poste partagé, proxy) | Moyenne | `Cache-Control: no-store` sur `/v1` |
| 7 | Exception « mot de passe à changer » trop large (expression non ancrée) | Moyenne | Expression ancrée, seules 4 routes autorisées |
| 8 | Algorithme JWT non épinglé | Moyenne | HS256 et émetteur imposés ; jetons `alg: none` ou falsifiés refusés (testé) |
| 9 | Dépendances vulnérables (multer, nodemailer, selfsigned/node-forge, Express) | Élevée | NestJS 10→11, Express 5, nodemailer 10, selfsigned 5. `npm audit --omit=dev` : 4 avis modérés restants (js-yaml via Swagger, uuid via exceljs), non exploitables dans cet usage |

| # | Faille | Gravité | Correctif |
|---|---|---|---|
| 10 | Mises à jour du poste Windows vérifiées seulement par empreinte fournie par le même serveur (serveur compromis = code arbitraire sur tous les postes) | Critique | Signature Ed25519 hors ligne : le poste refuse tout installateur non signé ou signé pour une autre version/empreinte (pas de rejeu). **Active dès que la clé publique est livrée dans l'application (voir §3)** |
| 11 | Application de bureau : ouverture de protocoles arbitraires, navigation hors origine, permissions web | Moyenne | Seuls http/https s'ouvrent à l'extérieur ; navigation externe bloquée ; permissions refusées |
| 12 | Android : sauvegarde cloud des données locales, trafic en clair possible, version livrée signée avec la clé de débogage | Élevée | `allowBackup=false`, `usesCleartextTraffic=false`, signature de production par secrets CI |

Vérifié sans faille : noms de tables dynamiques tous en liste blanche ; injections SQL testées (aucun effet) ; isolation entre coopératives (sécurité par ligne + tests croisés) ; contrôle du type réel des documents et empreinte SHA-256 ; réplication limitée à l'organisation du site ; élévation de privilèges par champs superflus (rejetée).

## 2. Mesures en place (rappel)

JWT court + jeton de renouvellement tournant, argon2id, permissions contrôlées côté serveur, sécurité par ligne PostgreSQL avec rôle applicatif non propriétaire, audit, écritures financières non modifiables, idempotence, validation stricte (liste blanche, champs inconnus refusés), helmet, métriques protégées par jeton, CORS restrictif, erreurs sans trace interne.

## 3. Actions à faire par vous (non automatisables)

1. **Clés de signature des mises à jour** : `node desktop/scripts/sign-release.js keygen <dossier-hors-dépôt>` ; copier la clé publique dans `desktop/src/release-public-key.pem`, reconstruire l'application ; garder la clé privée hors du dépôt (coffre). À chaque version : `node desktop/scripts/sign-release.js sign <installateur.exe> <version> <clé-privée.pem>` puis coller la signature dans l'écran Synchronisation au dépôt. Tant que la clé publique n'est pas livrée, la vérification de signature n'est pas imposée.
2. **Android** : créer un keystore de production (le conserver précieusement) et définir les secrets GitHub `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`, `ANDROID_KEY_PASSWORD`.
3. **Variables d'environnement de production** : `NODE_ENV=production`, `JWT_SECRET` long et aléatoire, `METRICS_TOKEN`, `ENABLE_API_DOCS` absent, `TRUST_PROXY` si derrière un proxy (sinon la limite par adresse voit l'adresse du proxy), `TLS_DIR` ou terminaison HTTPS en amont, `LOGIN_THROTTLE_LIMIT` laissé par défaut.
4. **Serveur local non initialisé** : tant que l'amorçage n'est pas fait, il est ouvert sur le réseau ; le démarrer avec `HOST=127.0.0.1` pour l'initialisation.
5. Sauvegardes chiffrées, disque chiffré, pare-feu (seuls 443 et SSH), mises à jour système.

## 4. Risques restants (honnêtement)

- Pas d'authentification à deux facteurs (à ajouter en priorité pour administrateurs et comptables).
- Aucun test d'intrusion indépendant ; aucun test sur appareils Android réels ni sur Windows réel.
- Le verrouillage par compte peut être utilisé pour empêcher un utilisateur de se connecter (déni de service ciblé) ; atténué par la limite par adresse.
- `/v1/mode` public révèle la version et la version du schéma.
- Secrets en variables d'environnement (pas de coffre) ; pas de chiffrement au repos des documents ni de la base (à fournir par l'hébergeur) ; pas d'antivirus sur les documents.
- Données locales du mobile non chiffrées et non purgées à la révocation.
- Signature des mises à jour inactive tant que la clé publique n'est pas livrée.
- Conformité aux textes sur les données personnelles à faire valider par un juriste.
- Les 4 avis `npm audit` modérés restants sont à suivre (Dependabot).

## 5. Procédure de révocation d'un accès (CDC §27.7)

1. `POST /users/:id/suspend` (permission `users:suspend`, motif obligatoire) : accès refusé dès la requête suivante.
2. Appareil perdu : `POST /users/:id/revoke-sessions` ; la synchronisation se bloque.
3. Changement de droits : `POST /users/:id/role`, effet immédiat.
4. Tout est tracé (`user_access_events`, audit) ; un administrateur actif doit toujours rester.

## 6. En cas d'incident

Suspendre les comptes concernés, révoquer les sessions, changer `JWT_SECRET` (déconnecte tout le monde), consulter l'audit et les journaux (`auth.login_while_locked`, échecs répétés), restaurer depuis la sauvegarde si nécessaire (`scripts/restore-test.sh`).